# TrustFedGNN: Auditing and Mitigating Report-Dependent Trust in Federated Graph Learning

Ngoc-Son-An Nguyen[1], Quang-Vinh Dang[2*], Tuong-Nguyen Huynh[1]

[1]Industrial University of Ho Chi Minh City, Ho Chi Minh City, Vietnam.
[2*]British University Vietnam, Hung Yen, Vietnam.

**Abstract**

Lenders and public agencies increasingly train graph neural networks jointly without pooling data. To keep such federations fair, several aggregation rules weight each client by the loss or disparity it reports about itself, which the server cannot verify. We audit this interface with a fixed-update intervention that changes only an adversary's report while client updates and server state stay constant. Across 30 seeds, all six report-dependent rules shift weight towards the adversary, by up to 0.91, whereas rules that read no report keep identical aggregates. We then propose TrustFedGNN, whose server rule, FU-Alignment, ignores client reports. It clips each update, scores it by projection onto a reference gradient from a small confidential server graph, and smooths the signed score over rounds. Under matched fairness poisoning, FU-Alignment lowers the adversary's mean weight relative to root-update FLTrust from 0.189 to 0.063 on German Credit and from 0.196 to 0.045 on Bail, with higher AUC on both datasets. An ablation attributes this gain to clipping and signed smoothing. Under adaptive reference evasion, FU-Alignment gives attackers the lowest weight and influence of four rules and retains the highest AUC; attackers that alternate honest and poisoning phases on German mark its boundary. On Pokec-z, its AUC is non-inferior to FedAvg's at a prespecified 0.01 margin. The study thus offers a reusable audit and an inspectable, report-independent aggregation rule with a mapped operating boundary.

**Keywords:** Federated Learning, Graph Neural Networks, Byzantine Robustness, Algorithmic Fairness, Trust Auditing

*Corresponding author: Quang-Vinh Dang (vinh.dq4@buv.edu.vn). Contributing authors: Ngoc-Son-An Nguyen (annns25871@pgr.iuh.edu.vn); Tuong-Nguyen Huynh (huynhtuongnguyen@iuh.edu.vn).

# 1 Introduction

Federated graph learning lets institutions train graph neural networks without pooling their subgraphs [1–4]. In credit and risk applications, however, collaboration must address both group disparities and malicious participants. Several fairness-aware aggregation rules weight clients using locally reported losses or fairness statistics [5–10]. Those statistics may be useful under honest participation, but the server cannot usually verify them without access to private client data. Consider a consortium of lenders that trains a credit-risk model on its members' customer graphs. If one member can raise its own aggregation weight by reporting a small disparity and a low loss, it can steer the shared model while appearing to be the most responsible participant, and no other member can detect this from the reports alone.

This paper asks a concrete operational question: what happens when an aggregation rule treats a self-reported scalar as evidence of client quality? We first separate that interface dependency from training dynamics. In a fixed-update intervention, client updates and server state remain identical while the adversary changes only its report. The resulting weight change measures the direct sensitivity of the rule to unverified telemetry. The audit identifies a manipulable input and measures how much weight each rule concedes through it; rules that read no report act as controls whose invariance confirms that the intervention isolates the report channel.

We then introduce TrustFedGNN and its server rule, FU-Alignment. The server holds a small confidential audit graph, computes task and fairness reference gradients on it, clips client updates, and weights them by signed, temporally smoothed projections onto the combined reference. The aggregation path consumes no client-reported loss or disparity. Server anchoring itself is established by FLTrust [11]; our contribution is the score dynamics (clipping before projection scoring and signed smoothing before rectification), whose effect we isolate by ablation, and an audit design that tests report dependence and adaptive-access boundaries.

The evidence supports three conclusions:

1. **Report-surface audit.** In a fixed-update audit, changing only the adversary's report changes its mean weight under six report-dependent rules, while CGSV, FLTrust, root-update FLTrust, and FU-Alignment produce bit-identical aggregates under a report falsified in every field except the sample count. An end-to-end run confirms the invariance for root-update FLTrust.
2. **Controlled mechanism evidence.** Under a controlled fairness-poisoning protocol, FU-Alignment assigns less adversarial weight than root-update FLTrust on German (0.0630 versus 0.1892) and Bail (0.0451 versus 0.1959), with higher AUC on both datasets in the same matched runs. A factorial ablation attributes the reduction to clipping and signed smoothing; the fairness term has no detectable effect at the weight used.
3. **Explicit boundaries.** An adaptive-access study shows that performance depends on dataset and adversarial access: under reference evasion FU-Alignment gives the adversaries the lowest weight and influence of the four rules compared and keeps the highest AUC on both datasets, and on Bail it has the lowest processed influence under sustained and intermittent poisoning; Section 4.4 identifies the access pattern

under which this advantage does not hold. A known-label control supports AUC non-inferiority to FedAvg on Pokec-z observed labels, but does not support fairness superiority.

***Relation to prior work***

FSER and FTGD adapt ideas from our earlier FedFairGNN study [10] and are held fixed across relevant comparisons. They are not contributions of this paper. The present contribution lies in the trust-surface audit, the FU-Alignment server rule, and the evidence-backed characterization of its operating boundary.

Section 2 positions the work against fairness-aware aggregation and server-anchored trust. Section 3 defines the threat model and method. Section 4 presents the evaluation, Section 5 discusses deployment and limitations, and Section 6 summarizes the supported conclusions.

# 2 Related Work and Setting

We review fairness-aware federated graph learning (Section 2.1), privacy limitations of reported fairness statistics (Section 2.2), Byzantine-robust aggregation and server-anchored trust (Section 2.3), client contribution evaluation (Section 2.4), and the architectural relation to prior work (Section 2.5).

## 2.1 Fairness in Federated Graph Learning

Graph neural networks [1, 2] trained under federated protocols [3, 4] enable collaborative inference over distributed subgraphs without pooling raw adjacency or feature matrices. Two principal paradigms address group fairness in this setting.

The first paradigm operates within client-side local training while leaving the global aggregation mechanism unmodified. FairGNN [12] employs adversarial training to purge sensitive demographic attributes from node representations, and FairSIN [13] neutralizes sensitive signals in feature space. Related methods rebalance graph topologies [14], learn invariant representations [15], debias input structures [16], optimize fair contrastive objectives [17], or disentangle sensitive latent factors [18, 19]. In vertical federations, FAVGNN [20] addresses feature-split graphs. Because client-side debiasing mechanisms execute locally, they provide no systemic defense if a participating client deliberately manipulates its training objective or falsifies its exported outputs.

The second paradigm intervenes at the aggregation stage, typically by weighting client updates according to self-reported scalars. FairFed [5] and FairGFL [21] reweight clients based on the gap between local reported fairness and global estimates; $F^2$GNN [7] and FedGraph-Fair [8] compute aggregation weights from reported graph-aware disparities or empirical loss values; PUFFLE [22] adapts client weights toward a target disparity; FedFACT [23] calibrates demographic trade-offs; $q$-FedAvg [6] powers client weights by reported empirical losses; and BFWA [10] adjusts client weights via bilevel optimization against reported disparity and utility metrics. PoPETs-FairFed [9] dampens FairFed weights polynomially but still consumes reported local disparity values. Across these rules, the aggregation surface depends critically on client-reported scalar metadata that the server cannot directly verify.

## 2.2 Privacy of Reported Fairness Statistics

Local differential privacy (LDP) [24, 25] is commonly considered to preserve client confidentiality when transmitting scalar statistics, and FedFDP [26] combines LDP with distributed fairness coordination. An extensive literature identifies fundamental trade-offs between differential privacy and algorithmic fairness, showing that private noise injection frequently degrades performance on demographic minority groups [27–30]. Cryptographic coordination, such as the homomorphic aggregation in PoPETs-FairFed [9], protects intermediate values in transit but does not attest to the truthfulness of local client computations. While prior studies investigate the fairness impact of private model training, our focus centers on the verification bottleneck: whether a central server can reliably enforce or audit fairness constraints when client reports are privatized under LDP.

## 2.3 Byzantine-Robust and Server-Anchored Aggregation

Classical Byzantine-robust aggregation mechanisms evaluate mutual distances among client updates in parameter space without requiring auxiliary server data. These include coordinate-wise median and trimmed mean [31], Krum and Multi-Krum [32], Bulyan [33], FoolsGold [34], and FLAME [35]. While effective against arbitrary unconstrained updates, distance-based filters remain blind to adversaries whose poisoned updates remain within the geometric spread of benign clients [36, 37].

A distinct lineage anchors aggregation against external reference directions. We examine three primary counterparts to delineate the novelty of FU-Alignment:

1. **CGSV** [38]: Gradient-driven rewards value uploaded gradients without an auxiliary dataset, using cosine-based contribution scores within the collaborative aggregation process. Its reference is therefore derived from participant updates. FU-Alignment instead scores against gradients computed on a server-held graph. We treat this as a difference in reference provenance, not as evidence that CGSV is generally insecure.
2. **FLTrust** [11]: FLTrust bootstraps trust from a server update computed on clean root data. It uses rectified cosine scores and normalizes client updates to the root-update norm, which limits update magnitude. FU-Alignment shares the server-anchoring principle but uses separate task and fairness gradients, cap-at-$C$ clipping, projection scores, and signed temporal smoothing. A factorial ablation (Section 4.3) attributes the measured difference to clipping and signed smoothing rather than to the fairness gradient.
3. **GuardFed** [39]: GuardFed maintains a fairness-aware reference model and evaluates client models using server-held clean or synthetic validation data. FU-Alignment instead uses first-order projections of clipped updates onto server-computed gradients and carries signed scores across rounds. Our experiments use a project-specific GuardFed-style adaptation, clearly labeled because it is not a reproduction of the original data and validation pipeline.

### 2.4 Evaluating Client Contributions

Shapley-value formulations quantify client value by computing average marginal improvements across client coalitions [40–42]. Exact computation requires evaluating $2^K$ model coalitions, and Monte Carlo approximations incur multiple evaluation passes per round. The score ledger in FU-Alignment provides an exact, first-order linear decomposition of per-round scores into utility and fairness components in $O(KP)$ time, offering inspectable auditability without claiming to approximate exponential cooperative-game Shapley values.

### 2.5 Relation to Prior Work

This work addresses the auditability and defense of aggregation rules in federated graph learning under report manipulation. FedFairGNN [10] introduced bilevel Frank–Wolfe optimization (BFWA) driven by reported client telemetry, paired with local client debiasing via fairness-sensitive edge reweighting (FSER) and fair targeted gradient descent (FTGD) on three-client fraud graphs. Here, BFWA is treated as an audited prior method. To isolate the server-side aggregation mechanism, our comparative evaluations adapt the FSER and FTGD client-side training mechanisms as a fixed local stack across all evaluated aggregation rules. These client-side components are held fixed and are not claimed as contributions of this study. The report-surface audit, scoped report-channel privacy analysis, FU-Alignment server rule, and adaptive-access evaluation constitute the contributions of this article.

## 3 Methodology

Section 3.1 states the setting, threat model, and assumptions. Section 3.2 gives two scoped observations that motivate server-side reference scoring. Section 3.3 defines FU-Alignment, and Sections 3.4 and 3.5 describe the shared client stack and audit ledger.

### 3.1 Setting, Threat Model and Assumptions

***Federated graph learning***

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X, A)$ be an attributed graph partitioned across $K$ clients. Client $k$ holds a subgraph $\mathcal{G}_k = (\mathcal{V}_k, \mathcal{E}_k, X_k, A_k)$ with disjoint node sets, and edges between different clients are not observed by anyone. Each node $v$ has a binary label $Y(v)$ and a binary sensitive attribute $S(v)$, such as gender or race. The clients jointly train the parameters $\theta \in \mathbb{R}^P$ of a graph neural network. Fairness is measured by the demographic parity disparity

$$\mathrm{DPD}(\theta) = \left| P(\hat{Y} = 1 \mid S = 0) - P(\hat{Y} = 1 \mid S = 1) \right|, \tag{1}$$

which a federation may require to satisfy

$$\mathrm{DPD}(\theta) \leq \tau \tag{2}$$

for a tolerance $\tau \geq 0$. We also report the equal opportunity disparity

$$\mathrm{EOD}(\theta) = \left| P(\hat{Y} = 1 \mid Y = 1, S = 0) - P(\hat{Y} = 1 \mid Y = 1, S = 1) \right|. \tag{3}$$

***Design goals***

An aggregation rule for this setting should (i) prevent a client from gaining weight through anything it declares about itself, (ii) limit poisoned-update influence in specified threat models, and (iii) preserve useful predictive performance in benign runs. Section 4 evaluates these goals without assuming that one rule dominates under every attack.

***Threat model***

Following Kerckhoffs' principle, the adversary knows the architecture, losses, and aggregation rule. It controls $f < K/2$ clients, which may send arbitrary updates and arbitrary reports. Section 4.1 evaluates three access schedules: sustained poisoning, intermittent poisoning that alternates honest and poisoning phases, and reference evasion that optimises an update against a surrogate of the server reference. Availability attacks, Sybil participation, and contamination of the server holdout remain outside the threat model.

***Assumptions on the server holdout***

The rule of Section 3.3 relies on a small labelled holdout graph $D_{\mathrm{root}}$ kept by the server. We make three assumptions about it; the adaptive-access study probes adaptive access but does not exhaustively test violations of these assumptions:

**A1** **Clean.** The labels and sensitive attributes of $D_{\mathrm{root}}$ are correct. In a cross-silo federation such a set can be an audit sample held by a regulator or curated by the consortium. FLTrust assumes a clean root dataset at the server [11], and GuardFed likewise keeps a small share of real labelled data there [39]; we additionally need the sensitive attribute of these nodes.

**A2** **Confidential.** Clients can read neither $D_{\mathrm{root}}$ nor the reference gradient computed on it.

**A3** **Representative.** $D_{\mathrm{root}}$ approximates the target population well enough for its task and fairness gradients to be useful references.

We do not consider cryptographic attestation, trusted hardware at every client, or incentive mechanisms that would make the reports themselves trustworthy. The rule below requires none of them at the clients, and the server may keep its holdout in a trusted execution environment to support A2.

## 3.2 Analytical Motivation

Two observations motivate a reference computed by the server: report-free weights are invariant to report falsification, and update similarity alone need not reveal a small orthogonal perturbation. Neither observation establishes general robustness.

### *Rules that read reports*

The weights of a report-reading rule depend on numbers that clients choose. The converse is immediate and worth stating because Section 4.2 verifies it directly.

**Remark 1** (Rules that read no report) If the weights depend only on the client updates and on quantities computed by the server, that is, $\boldsymbol{w} = \mathcal{F}(\{\Delta_k\}_{k=1}^K, g_{\text{target}})$, then

$$\frac{\partial w_k}{\partial \widehat{\text{DPD}}_j} \equiv 0 \qquad \forall\, k, j \in [K], \tag{4}$$

so no falsified report can change them. FLTrust and the rule of Section 3.3 have this property by construction, and CGSV reads no disparity report. It is necessary but not sufficient for robustness: a reference derived from the client updates, as in CGSV, can still be moved by a coordinated group of clients, and a task-only server reference, as in FLTrust, carries no information about fairness.

A natural repair for a report-reading rule is to privatise the reports. Each client then releases its two group means $\mu_{k,a}$, the mean soft prediction of group $a$, with Gaussian noise:

$$\tilde{\mu}_{k,a} = \mu_{k,a} + \xi_{k,a}, \quad \xi_{k,a} \sim \mathcal{N}\left(0, \sigma_k^2\right), \quad \sigma_k = z\, \Delta_{\boldsymbol{\mu},k}, \tag{5}$$

where $\Delta_{\boldsymbol{\mu},k} = \sqrt{1/n_{k,0}^2 + 1/n_{k,1}^2}$ is the $\ell_2$ sensitivity of $(\mu_{k,0}, \mu_{k,1})$ and $z$ is the smallest noise multiplier for which Rényi accounting [25] over all $R \cdot E$ releases of a run gives $(\epsilon, \delta) = (8, 10^{-5})$; for example, $z = 4.94$ for $R = 20$, $E = 3$. The reported disparity is $\widetilde{\text{DPD}}_k = |\tilde{\mu}_{k,0} - \tilde{\mu}_{k,1}|$. Section S2 of the Supplementary Information (ESM 1) shows that this repair has two defects. Because the reported disparity is an absolute value, Gaussian noise biases it upwards even when a client satisfies demographic parity, and the bias accumulates across clients instead of cancelling. In addition, a two-point Le Cam argument shows that a single round of $K$ reports cannot reliably separate clients with disparity at most $\tau/2$ from clients with disparity at least $3\tau/2$ unless $\tau\sqrt{K}$ is large relative to the effective noise level. Both statements concern one round of independent Gaussian releases and the report channel only; they motivate a reference that the server computes itself rather than a general impossibility result for private fairness mechanisms.

Our earlier rule BFWA enforces the constraint by dual steps on such reports. Its response in Section 4.2 identifies sensitivity of that reporting channel rather than a defect in its honest-report optimization.

### *Detectors that compare updates*

Rules that read no report, such as Krum or FLAME, judge each update by its distance to the others. A perturbation that changes only fairness can stay close to the benign updates under such a distance.

**Proposition 1** (Cosine distance cannot separate fairness-only perturbations) *Let* $\mathbb{R}^P = \mathcal{S}_{\text{task}} \oplus \mathcal{S}_{\text{fair}}$ *with* $\mathcal{S}_{\text{fair}} = \mathcal{S}_{\text{task}}^{\perp}$*, and consider a detector that thresholds the cosine distance* $\mathcal{D}_{\cos}(u_i, u_j) = 1 - \frac{\langle u_i, u_j \rangle}{\|u_i\|_2 \|u_j\|_2}$ *at* $\rho_{\text{thresh}} \in (0, 1)$*. Then for any benign update* $u_{\text{benign}} \in \mathcal{S}_{\text{task}}$ *and any* $\delta_{\text{fair}} \in \mathcal{S}_{\text{fair}}$ *with* $\|\delta_{\text{fair}}\|_2 < \sqrt{\rho_{\text{thresh}}/(1-\rho_{\text{thresh}})}\, \|u_{\text{benign}}\|_2$*, the update* $u_{\text{adv}} = u_{\text{benign}} + \delta_{\text{fair}}$ *satisfies*

$$\mathcal{D}_{\cos}(u_{\text{adv}}, u_{\text{benign}}) \leq \frac{\|\delta_{\text{fair}}\|_2^2}{\|u_{\text{benign}}\|_2^2 + \|\delta_{\text{fair}}\|_2^2} < \rho_{\text{thresh}}, \tag{6}$$

*so the detector cannot separate* $u_{\text{adv}}$ *from* $u_{\text{benign}}$*.*

*Proof* Because $\langle u_{\text{benign}}, \delta_{\text{fair}} \rangle = 0$,

$$\begin{aligned}
\mathcal{D}_{\cos}(u_{\text{adv}}, u_{\text{benign}}) &= 1 - \frac{\|u_{\text{benign}}\|_2^2}{\sqrt{\|u_{\text{benign}}\|_2^2 + \|\delta_{\text{fair}}\|_2^2}\, \|u_{\text{benign}}\|_2} \\
&= 1 - \left(1 + \frac{\|\delta_{\text{fair}}\|_2^2}{\|u_{\text{benign}}\|_2^2}\right)^{-1/2}.
\end{aligned}$$

The inequality $1 - 1/\sqrt{1+x} \leq x/(1+x)$ gives the first inequality of Eq. (6), and the condition on $\|\delta_{\text{fair}}\|_2$ is equivalent to $x/(1+x) < \rho_{\text{thresh}}$ for $x = \|\delta_{\text{fair}}\|_2^2 / \|u_{\text{benign}}\|_2^2$. □

**Remark 2** (Scope) The proposition says that such a perturbation cannot be detected, not how much weight it receives or how much disparity it adds; Section 4 measures both. That benign updates lie, at least approximately, in $\mathcal{S}_{\text{task}}$ is an idealisation that we did not measure, and $\mathcal{S}_{\text{fair}}$ is defined geometrically, without fairness semantics of its own. The result applies to detectors that threshold cosine distance, such as the clustering in FLAME, and not to Krum and Multi-Krum, which use Euclidean distances.

## 3.3 FU-Alignment: Scoring Updates Against a Server Reference

FU-Alignment replaces the reports by a reference that only the server can compute (Figure 1). In words: the server shrinks every update to norm at most $C$, measures to first order how much the update would lower the server's loss on its holdout, keeps a running average of that measurement for each client, and averages the clipped updates with weights proportional to the positive part of the running average.

### *Server reference*

Before the clients are formed, the server sets aside up to 200 nodes of the label-stratified validation split, and never more than half of it, as its holdout $D_{\text{root}}$ (125 nodes on German Credit and 200 on the larger graphs). No client owns these nodes, and no edge connects them to a client subgraph. In round $t$ the server evaluates two

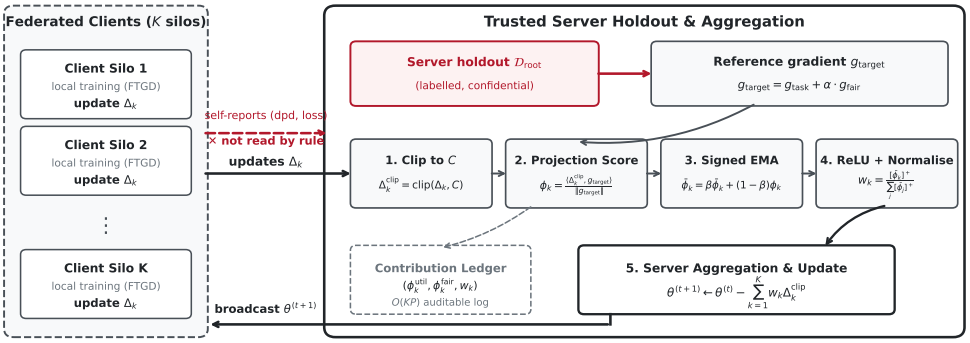



**Fig. 1 FU-Alignment.** Clients send only their updates; the reports that report-reading rules depend on are not read. The server computes a reference gradient on its own holdout graph, clips each update, scores it by its projection onto the reference, smooths the scores over rounds, and averages the clipped updates with weights proportional to the positive part of the smoothed scores. The two parts of each score are logged for audit.

gradients on $D_{\text{root}}$ at the current global model:

$$g_{\text{task}}^{\text{srv}} = \nabla_\theta \mathcal{L}_{\text{task}}(\theta^{(t)}; D_{\text{root}}), \qquad g_{\text{fair}}^{\text{srv}} = \nabla_\theta \mathcal{L}_{\text{fair}}^{\text{srv}}(\theta^{(t)}; D_{\text{root}}),$$
$$\mathcal{L}_{\text{fair}}^{\text{srv}} = (\mu_0^{\text{srv}} - \mu_1^{\text{srv}})^2, \tag{7}$$

where $\mu_a^{\text{srv}}$ is the mean soft prediction for group $a$ on $D_{\text{root}}$. The squared gap is differentiable at zero, and the server computes it on its own data without noise. The reference combines the two gradients with a fixed weight $\alpha \geq 0$:

$$g_{\text{target}}^{(t)} = g_{\text{task}}^{\text{srv}} + \alpha\, g_{\text{fair}}^{\text{srv}}. \tag{8}$$

The fairness term tilts the reference towards lower disparity on the holdout. It does not filter clients by itself, and at $\alpha = 0.1$ its measured effect on allocation is negligible (Section 4.3).

### *Scoring and aggregation*

Each client update $\Delta_k^{(t)} = \theta^{(t)} - \theta_k^{(t)}$ is first clipped to norm $C$,

$$\Delta_k^{\text{clip}(t)} = \Delta_k^{(t)} \cdot \min\left(1, \frac{C}{\left\|\Delta_k^{(t)}\right\|_2}\right), \tag{9}$$

and then scored by its projection onto the reference,

$$\varphi_k^{(t)} = \frac{\langle \Delta_k^{\text{clip}(t)}, g_{\text{target}}^{(t)} \rangle}{\left\|g_{\text{target}}^{(t)}\right\|_2}. \tag{10}$$

An update that would raise the server's holdout objective, such as a sign-flipped one, receives a negative score. The signed scores are smoothed over rounds,

$$\bar{\varphi}_k^{(0)} = \varphi_k^{(0)}, \qquad \bar{\varphi}_k^{(t)} = \beta\, \bar{\varphi}_k^{(t-1)} + (1-\beta)\, \varphi_k^{(t)} \quad (t \geq 1), \tag{11}$$

and only then rectified and normalised,

$$w_k^{(t)} = \frac{\mathrm{ReLU}(\bar{\varphi}_k^{(t)}) \cdot \mathbb{I}(\left\|\Delta_k^{(t)}\right\| > 0)}{\sum_{j=1}^{K} \mathrm{ReLU}(\bar{\varphi}_j^{(t)}) \cdot \mathbb{I}(\left\|\Delta_j^{(t)}\right\| > 0)}, \tag{12}$$

with uniform weights over the clients with a non-zero update if no smoothed score is positive. The global model moves by the weighted clipped updates:

$$\theta^{(t+1)} = \theta^{(t)} - \sum_{k=1}^{K} w_k^{(t)} \Delta_k^{\mathrm{clip}(t)}. \tag{13}$$

The canonical FU-Alignment cells use $\alpha = 0.1$, $\beta = 0.9$, and $C = 10$; campaign manifests record any campaign-specific variation.

***Design rationale***

Each element answers a weakness identified in Section 3.2; Sections 4.4 and 4.3 evaluate the resulting rule as a whole.

(i) **The reference does not depend on the clients.** Its task and fairness signals are computed on server-held data.

(ii) **The score bounds magnitude and keeps direction.** Clipping before scoring caps what a single update can contribute to its score and to the aggregate (Proposition 2), while the projection retains its component along the reference. The factorial ablation in Section 4.3 shows that replacing clipping by rescaling increases adversarial weight.

(iii) **The score remembers its sign.** Because the signed score is smoothed before it is rectified, a client that turns against the reference loses its weight within a bounded number of rounds (Proposition 2), and a client that has opposed the reference must point towards it for several rounds before it regains weight.

Table 1 sets these choices against FLTrust and GuardFed, the two server-anchored rules closest to ours.

**Proposition 2** (Bounded influence) *Let the weights be given by Eq. (12), including the uniform fallback, and the global update by Eq. (13).*

*(i) Whatever finite updates the clients send,* $\left\|\theta^{(t+1)} - \theta^{(t)}\right\|_2 \leq C$.

**Table 1 FU-Alignment against the closest server-anchored rules.** The three rules all score clients against data held by the server; they differ in what the score measures, how update magnitude is handled, and whether scores carry memory. FLTrust and GuardFed as described in [11, 39].

| | **FLTrust** | **GuardFed** | **FU-Alignment** |
|---|---|---|---|
| Reference | server model update on root data | update of a reference model trained on server and synthetic data with fairness reweighting | task gradient plus $\alpha$ times a fairness gradient on the holdout |
| Client score | ReLU of the cosine with the reference | ReLU of the cosine, times $e^{-\tau_G g_k}$, where $g_k$ is the equal-opportunity gap of the client model | projection of the clipped update onto the reference |
| Update magnitude | every update rescaled to the reference norm | not modified | clipped to norm $C$ |
| Memory | none | none | EMA of the signed score, before rectification |
| Weights | proportional to the score | uniform over clients whose score exceeds $\Gamma_G$ | proportional to the positive part of the smoothed score |
| Server work per round | one update on root data and $K$ cosines | a reference update, $K$ cosines and $K$ model evaluations | two gradients on the holdout and $K$ inner products |

*(ii) Suppose that client $k$ has $\varphi_k^{(t)} \le -m$ for some $m > 0$ and all $t \ge t_0 \ge 1$. Then $\bar{\varphi}_k^{(t)} \le 0$, and hence $w_k^{(t)} = 0$ whenever the fallback is not used, for all $t \ge t_0 + s - 1$ with*

$$s = \left\lceil \frac{\log\left((C+m)/m\right)}{\log(1/\beta)} \right\rceil . \tag{14}$$

*Proof* (i) In both branches the weights are non-negative and sum to one, and $\left\|\Delta_k^{\mathrm{clip}(t)}\right\|_2 \le C$, so $\left\|\sum_k w_k^{(t)} \Delta_k^{\mathrm{clip}(t)}\right\|_2 \le \sum_k w_k^{(t)} \left\|\Delta_k^{\mathrm{clip}(t)}\right\|_2 \le C$. (ii) By the Cauchy–Schwarz inequality $|\varphi_k^{(t)}| \le \left\|\Delta_k^{\mathrm{clip}(t)}\right\|_2 \le C$ for every $t$, and therefore $\bar{\varphi}_k^{(t)} \le C$, since it is a convex combination of scores. Unrolling Eq. (11) for $n \ge 1$ rounds from $t_0 - 1$ gives $\bar{\varphi}_k^{(t_0-1+n)} = \beta^n \bar{\varphi}_k^{(t_0-1)} + (1-\beta)\sum_{j=0}^{n-1} \beta^j \varphi_k^{(t_0-1+n-j)} \le \beta^n C - m(1-\beta^n)$, which is non-positive as soon as $\beta^n \le m/(C+m)$, that is, for all $n \ge s$. A non-positive smoothed score has zero rectified value, so the numerator of Eq. (12) vanishes. □

Part (i) means that no update, however large, can move the global model by more than $C$ in one round. Part (ii) means that a client that persistently opposes the reference loses its weight after at most $s$ rounds. If it opposes the reference in every round from the first one, Eq. (11) gives $\bar{\varphi}_k^{(t)} \le -m$ in every round by induction, so its weight is zero from the start whenever the fallback is not used; the delay of $s$ rounds applies only to a client that first builds up positive history, and for small $m$ it can exceed the length of our runs. Neither part bounds the weight of an adversary whose update is aligned with the reference; Section 4 measures that case.

**Algorithm 1** One round of FU-Alignment

**Require:** Global model $\theta^{(t)}$, holdout $D_{\text{root}}$, smoothed scores $\bar{\varphi}^{(t-1)}$, $\alpha = 0.1$, $\beta = 0.9$, $C = 10$.
**Ensure:** $\theta^{(t+1)}$, $\bar{\varphi}^{(t)}$ and the ledger entries of round $t$.

Compute $g^{\text{srv}}_{\text{task}}$ and $g^{\text{srv}}_{\text{fair}}$ on $D_{\text{root}}$ and set $g^{(t)}_{\text{target}} \leftarrow g^{\text{srv}}_{\text{task}} + \alpha g^{\text{srv}}_{\text{fair}}$ — Eqs. (7)–(8)
Broadcast $\theta^{(t)}$; each client $k$ trains locally (Section 3.4) and returns $\Delta^{(t)}_k$
**for** each client $k \in [K]$ **do**
  $\Delta^{\text{clip}(t)}_k \leftarrow \Delta^{(t)}_k \cdot \min(1, C/\left\|\Delta^{(t)}_k\right\|_2)$ — Eq. (9)
  $\varphi^{(t)}_k \leftarrow \langle \Delta^{\text{clip}(t)}_k, g^{(t)}_{\text{target}} \rangle / \left\|g^{(t)}_{\text{target}}\right\|_2$ — Eq. (10)
  $\bar{\varphi}^{(t)}_k \leftarrow \varphi^{(t)}_k$ if $t = 0$, else $\beta \bar{\varphi}^{(t-1)}_k + (1-\beta)\varphi^{(t)}_k$ — Eq. (11)
**end for**
Compute $w^{(t)}$ by Eq. (12), with the uniform fallback over clients with $\Delta^{(t)}_k \neq \mathbf{0}$
$\theta^{(t+1)} \leftarrow \theta^{(t)} - \sum_k w^{(t)}_k \Delta^{\text{clip}(t)}_k$ — Eq. (13)
Log $\langle t, k, w^{(t)}_k, \varphi^{\text{util}}_k, \varphi^{\text{fair}}_k \rangle$ for every $k$ — Eq. (19)

***Server requirements and cost***

The server needs a holdout of at most 200 labelled nodes with their sensitive attribute. In each round it evaluates two gradients on the holdout, clips the $K$ updates and computes $K$ inner products, which costs $O(KP)$ in addition to the two gradient evaluations. The clients run standard local training and need to send only their updates.

## 3.4 Client-Side Training Stack

To isolate the server-side aggregation mechanism, all methods evaluated in this paper use a common local training stack that adapts the FSER and FTGD ideas from our prior work [10]. These fixed client-side components are not claimed as contributions of this study. TrustFedGNN's aggregation path consumes model updates and server-computed reference signals without using client-reported fairness or performance statistics.

The local backbone is a two-layer GAT [2] with hidden dimension 64, batch normalisation, and residual connections, combined with fairness-sensitive edge reweighting (FSER), which initially attenuates inter-group edges by a factor $\gamma_{\text{init}} = 0.5$. Local training adapts fair targeted gradient descent (FTGD). Each client computes a task gradient and the gradient of a fairness loss $\mathcal{L}_{\text{fair}} = |\mu_{k,0} - \mu_{k,1}|$ built from its group means; when report privatisation is enabled, the released means $\tilde{\mu}_{k,a}$ of Eq. (5) replace them and are computed by a forward pass that does not take $S$ as input:

$$g_{k,\text{task}} = \nabla_\theta \mathcal{L}_{\text{task}}(\theta; \mathcal{G}_k), \qquad g_{k,\text{fair}} = \nabla_\theta \mathcal{L}_{\text{fair}}(\theta; \mathcal{G}_k). \tag{15}$$

It removes from the task gradient its component along the fairness gradient,

$$g^{\perp}_k = \begin{cases} g_{k,\text{task}} - \dfrac{\langle g_{k,\text{task}}, g_{k,\text{fair}} \rangle}{\|g_{k,\text{fair}}\|^2} g_{k,\text{fair}}, & \|g_{k,\text{fair}}\| \geq 10^{-8}, \\ g_{k,\text{task}}, & \text{otherwise}, \end{cases} \tag{16}$$

so that $\langle g_k^{\perp}, g_{k,\text{fair}} \rangle = 0$, and steps along

$$u_k = g_k^{\perp} + g_{k,\text{fair}}. \tag{17}$$

To run every rule under a common protocol, each client also exports a report containing its training-set size, validation AUC, loss, disparity, and equal opportunity gap; when privatisation is enabled, the reported disparity is the privatised one. FU-Alignment and FLTrust never read this report; CGSV reads only the sample count in a fallback branch, while the report exists for the rules that read it. Privatisation therefore protects a channel that FU-Alignment does not use.

### 3.5 Contribution Ledger

Because the reference is a sum of two gradients, each score splits exactly into a utility part and a fairness part.

**Proposition 3** (Linear decomposition) *For every client and round,*

$$\varphi_k^{(t)} = \varphi_k^{\text{util}} + \varphi_k^{\text{fair}}, \qquad \varphi_k^{\text{util}} = \frac{\langle \Delta_k^{\text{clip}(t)}, g_{\text{task}}^{\text{srv}} \rangle}{\left\| g_{\text{target}}^{(t)} \right\|_2}, \quad \varphi_k^{\text{fair}} = \frac{\alpha \langle \Delta_k^{\text{clip}(t)}, g_{\text{fair}}^{\text{srv}} \rangle}{\left\| g_{\text{target}}^{(t)} \right\|_2}, \tag{18}$$

*and both parts cost $O(KP)$ per round.*

*Proof* The inner product is linear in its second argument and $g_{\text{target}}^{(t)} = g_{\text{task}}^{\text{srv}} + \alpha g_{\text{fair}}^{\text{srv}}$; dividing by the common normaliser preserves the identity. □

In every round the server appends

$$\mathcal{T}_k^{(t)} = \left\langle t, k, w_k^{(t)}, \varphi_k^{\text{util}}, \varphi_k^{\text{fair}} \right\rangle \tag{19}$$

to a round log, which records how much of each client's score came from utility and how much from fairness. The implementation stores both parts before the division by $\left\| g_{\text{target}}^{(t)} \right\|_2$, which leaves their ratio unchanged. By construction of Eq. (12), the weights lie on the probability simplex and a client that sends a zero update receives zero weight. The decomposition is a first-order attribution and does not approximate Shapley values.

## 4 Experimental Design and Results

We organize the evaluation around four questions. RQ1 asks whether changing only a client report changes its aggregation weight. RQ2 asks whether FU-Alignment removes that dependency by construction. RQ3 compares FU-Alignment with a faithful root-update FLTrust control under a matched fairness-poisoning protocol and asks which of its components matter. RQ4 tests where the observed behavior persists or fails under different adversarial access schedules and a corrected label protocol. The studies use

different federation sizes and horizons (Table 2), so numbers are compared only within a study.

## 4.1 Experimental Design

***Datasets***

German Credit (1,000 nodes, 21,742 edges, sensitive attribute gender) and Bail Recidivism (18,876 nodes, 311,870 edges, race) follow the NIFTY preprocessing [17]; Pokec-z (67,796 nodes, 617,958 edges, region) follows FairGNN [12, 43]. On Pokec-z we evaluate only the 10,262 nodes with an observed label (15.1%) rather than treating missing labels as negatives. Before partitioning, the server removes its holdout $D_{\text{root}}$ from the label-stratified validation split: $\min(200, |\mathcal{V}_{\text{val}}|/2)$ nodes, which is 125 nodes on German and 200 on Bail and Pokec-z. The remaining nodes are split across $K$ clients by a Dirichlet label skew with concentration 0.3; edges between clients and edges to the holdout are dropped.

***Training***

Clients train full-batch for three local epochs per round with AdamW (learning rate 0.01, weight decay $10^{-5}$). Except in the two report audits, every method uses the same client stack (Section 3.4): a two-layer GAT with FSER and FTGD. Because the audits test only the aggregation interface, they hold a two-layer GCN client trained with FTGD fixed across all rules; the fixed-update audit uses a 100-node holdout and the end-to-end audit the default holdout above. Report privatisation is enabled only in the Pokec-z control ($\epsilon = 8$) and in the report-privacy diagnostic; elsewhere FTGD uses exact group means. FU-Alignment uses $\alpha = 0.1$, $\beta = 0.9$, and $C = 10$. The factorial cells and the adaptive-access study use a cosine variant of the score, $\cos(\Delta_k^{\text{clip}}, g_{\text{target}})/\left\|g_{\text{target}}\right\|_2$, in place of the projection of Eq. (10); in the controlled study its adversarial weight is close to that of the projection score (0.074 versus 0.063 on German, SR+C cell of Table 5). Because this score is not bounded by $C$, Proposition 2(ii) does not apply to it directly.

***Attacks***

In *fairness poisoning*, each adversarial client trains on its own data to minimise $\mathcal{L}_{\text{BCE}} - 10\,\widehat{\text{DPD}}_{\text{soft}}$, that is, to increase disparity while keeping some predictive signal, and reports $\widehat{\text{DPD}} = 0$ with near-perfect utility. The fixed-update audit uses the same update but switches only the report between honest and falsified values. The adaptive-access study varies when and how the adversary acts over 40 rounds:

- **Sustained poisoning:** honest in rounds 0–9, fairness poisoning from round 10 onward.
- **Intermittent poisoning:** honest in rounds 0–9, then alternating five poisoning rounds and five honest rounds, so that honest phases can rebuild the adversary's score history.
- **Reference evasion:** from round 1 the adversaries send one crafted update, initialised at the adversary's honest update and optimised for 50 steps to align with

**Table 2 Evaluation protocol.** Each study answers one research question with its own federation size and number of rounds $R$; results are compared only within a study. $K$ is the number of clients and $f$ the number of adversaries. The last column gives the identifier of the corresponding artifact in the code repository.

| Study | RQ | Data | $K/f$ | $R$ | Seeds | Rules | Artifact |
|---|---|---|---|---|---|---|---|
| Fixed-update audit | 1, 2 | German | 5 / 1 | 20 | 30 | 10 rules (Table 3) | C20r |
| End-to-end audit | 1 | German, Bail | 5 / 1 | 20 | 30 | root-FLTrust | C20b/c |
| Controlled fairness poisoning | 3 | German, Bail | 5 / 1 | 20 | 30 | FU-Alignment, root-FLTrust, FedAvg; factorial; $\alpha = 0$ | C23, C23b, C26 |
| Adaptive access | 4 | German, Bail | 10 / 2 | 40 | 10 | FedAvg, root-FLTrust, FU-Alignment, GuardFed-style | C24 |
| Known-label control | 4 | Pokec-z | 10 / 0 | 50 | 10 | FedAvg, FU-Alignment | C25 |

the previous global step (a public surrogate of the server reference) while opposing the adversary's own fairness gradient, with a penalty on any norm above five times that of the honest update. The adversary never sees $D_{\text{root}}$.

### *Comparators and metrics*

The audit covers six report-dependent rules (FairFed, $q$-FFL-eq, $F^2$GNN, FedGraph-Fair, PoPETs-FairFed, and BFWA) and four rules that do not read the audited report (CGSV, FLTrust, root-update FLTrust, and FU-Alignment). Root-update FLTrust computes its reference update on the same server holdout as FU-Alignment and is the faithful control in every later study. The adaptive-access study adds FedAvg and a GuardFed-style comparator whose two hyperparameters were selected on separate tuning seeds at $K = 5$ with one adversary and then frozen and reused at $K = 10$ (Section S1 of ESM 1); we label it as an adaptation because the holdout and implementation details differ from the original [39]. We report AUC, DPD, and EOD on the test split; DPD and EOD use hard predictions at a 0.5 threshold. The adversarial weight $w_{\text{adv}}$ is the total aggregation weight of the adversarial clients, averaged over rounds (in the adaptive-access study, over the rounds in which the adversaries poison). The processed influence $I_{\text{adv}} = \| \sum_{i \in \mathcal{A}} w_i \tilde{\Delta}_i \|_2$, also averaged over rounds, measures how far the adversarial contributions move the model after each rule's own clipping or rescaling $\tilde{\Delta}_i$.

### *Statistics*

Seeds are paired within each study: the partition, holdout, and initialisation are identical across rules for a given seed. We report paired mean differences with 95% bootstrap confidence intervals and two-sided Wilcoxon signed-rank tests, with Holm correction across the eight dataset–metric contrasts against root-update FLTrust and across those of the $\alpha = 0$ comparison; contrasts against FedAvg are reported unadjusted. The Pokec-z control tests non-inferiority against a margin of 0.01 AUC, fixed before the run, using the one-sided 95% bootstrap lower bound.

## 4.2 RQ1: Does Falsified Telemetry Change Trust Allocation?

The fixed-update audit replays the same client updates, server state and server reference twice and changes only the adversary's report. Report-dependent rules receive the lie that best serves the adversary under each rule; the report-free rules receive a report in which every field except the sample count is falsified. This design tests the aggregation interface rather than the success of a jointly trained attack.

**Table 3 Fixed-update intervention isolates sensitivity to a falsified client report.** In both arms the client updates, server state and server reference are identical; only the adversary's report changes. Report-dependent rules receive their best-response lie; the report-free rules below the line receive a report in which every field except the sample count is falsified. Means over 30 seeds and 20 rounds on German Credit. The final column counts seeds with a nonzero change.

| Aggregation rule | $w_{\text{adv}}^{\text{honest}}$ | $w_{\text{adv}}^{\text{false}}$ | $\Delta w_{\text{adv}}$ | Nonzero |
|---|---|---|---|---|
| FairFed | 0.0703 | 0.2387 | 0.1684 | 30/30 |
| $q$-FFL-eq | 0.1999 | 0.5050 | 0.3051 | 30/30 |
| $F^2$GNN | 0.0718 | 0.9802 | 0.9084 | 30/30 |
| FedGraph-Fair | 0.2210 | 0.2605 | 0.0395 | 26/30 |
| PoPETs-FairFed | 0.1773 | 0.1854 | 0.0080 | 30/30 |
| BFWA | 0.0707 | 0.7204 | 0.6497 | 25/30 |
| CGSV | 0.1918 | 0.1918 | 0.0000 | 0/30 |
| FLTrust | 0.3209 | 0.3209 | 0.0000 | 0/30 |
| root-update FLTrust | 0.1624 | 0.1624 | 0.0000 | 0/30 |
| FU-Alignment | 0.1079 | 0.1079 | 0.0000 | 0/30 |

Every audited report-dependent rule responds in at least 25 of 30 seeds, whereas CGSV, both FLTrust variants, and FU-Alignment do not respond in any seed: their weights and their aggregate are bit-identical under the honest and the falsified report (Table 3). The magnitude varies sharply across rules; the mean adversarial-weight change is 0.908 for $F^2$GNN, 0.650 for BFWA, and 0.305 for $q$-FFL-eq. These values measure an input dependency of the evaluated implementations.

An end-to-end audit, in which the adversary trains jointly with the federation and falsifies its report in every round, confirms the invariance for root-update FLTrust: its mean adversarial weight is the same with and without the lie (0.143 on German, 0.129 on Bail). A separate diagnostic of privatised reports, confined to the scalar report channel and BFWA's decision, is in Section S2 of ESM 1.

## 4.3 RQ2–RQ3: Report Independence and Controlled Efficacy

FU-Alignment never reads the audited client telemetry, so the fixed-update audit verifies report independence for the implemented aggregation path. The controlled study then asks a different question: how does that path behave under matched fairness poisoning when compared with root-update FLTrust?

FU-Alignment assigns lower mean adversarial weight than root-update FLTrust on both datasets: 0.0630 versus 0.1892 on German (lower in 29 of 30 paired seeds) and 0.0451 versus 0.1959 on Bail (27 of 30) (Table 4). It also has higher AUC on both datasets (+0.021 and +0.024). With Holm correction across the eight dataset–metric contrasts, both allocation reductions, both AUC gains, and the German EOD reduction remain significant; the DPD contrasts and the Bail EOD contrast do not. Against FedAvg in the same runs, FU-Alignment lowers adversarial weight, DPD, and EOD on both datasets, at the cost of 0.015 AUC on Bail. A logged replay reproduces these runs bit for bit and records per-round weights, scores, and $I_{\text{adv}}$; FU-Alignment's

**Table 4 Controlled fairness-poisoning comparison.** Panel A gives means over 30 paired seeds ($K = 5$, one adversary). Panel B gives FU-Alignment minus comparator with paired 95% bootstrap confidence intervals. Against root-update FLTrust, Holm correction across the eight dataset–metric tests retains $w_{\mathrm{adv}}$ and AUC on both datasets and EOD on German; neither DPD contrast nor Bail EOD is significant. Against FedAvg, $w_{\mathrm{adv}}$, DPD and EOD decrease on both datasets at unadjusted $p < 0.05$, while Bail AUC decreases. DPD and EOD use a 0.5 decision threshold.

**A. Endpoint means**

| Dataset | Rule | $w_{\mathrm{adv}}$ | AUC | DPD | EOD |
|---|---|---|---|---|---|
| German | FedAvg | 0.1649 | 0.6260 | 0.1550 | 0.1626 |
| German | root-update FLTrust | 0.1892 | 0.6205 | 0.1165 | 0.1361 |
| German | FU-Alignment | 0.0630 | 0.6414 | 0.0720 | 0.0688 |
| Bail | FedAvg | 0.1939 | 0.7381 | 0.0366 | 0.0546 |
| Bail | root-update FLTrust | 0.1959 | 0.6990 | 0.0265 | 0.0495 |
| Bail | FU-Alignment | 0.0451 | 0.7234 | 0.0197 | 0.0361 |

**B. Paired difference: FU-Alignment minus comparator**

| Dataset | Comparator | $\Delta w_{\mathrm{adv}}$ [95% CI] | Δ AUC [95% CI] | Δ DPD [95% CI] | Δ EOD [95% CI] |
|---|---|---|---|---|---|
| German | root-FLTrust | −0.126 [−0.149, −0.103] | +0.021 [+0.006, +0.036] | −0.045 [−0.084, −0.010] | −0.067 [−0.114, −0.025] |
| German | FedAvg | −0.102 [−0.164, −0.047] | +0.015 [−0.001, +0.034] | −0.083 [−0.146, −0.027] | −0.094 [−0.170, −0.029] |
| Bail | root-FLTrust | −0.151 [−0.185, −0.114] | +0.024 [+0.005, +0.043] | −0.007 [−0.019, +0.004] | −0.013 [−0.027, +0.000] |
| Bail | FedAvg | −0.149 [−0.201, −0.103] | −0.015 [−0.027, −0.002] | −0.017 [−0.033, −0.004] | −0.018 [−0.033, −0.005] |

**Table 5 Factorial mechanism ablation under fairness poisoning.** Mean adversarial weight for smooth–rectify (SR) or rectify–smooth (RS), combined with clipping (C) or rescaling (R), over 30 paired seeds. All four cells score updates by $\cos(\Delta_k, g)/\|g\|$. Positive factorial effects mean that rectify-first or rescaling increases adversarial allocation. Brackets are paired 95% bootstrap confidence intervals.

| Dataset | SR+C | SR+R | RS+C | RS+R | Order effect | Magnitude effect |
|---|---|---|---|---|---|---|
| German | 0.074 | 0.157 | 0.115 | 0.186 | +0.035 [+0.022,+0.049] | +0.077 [+0.053,+0.099] |
| Bail | 0.046 | 0.080 | 0.078 | 0.097 | +0.024 [+0.016,+0.034] | +0.027 [+0.018,+0.036] |

mean processed influence is 0.401 against 1.172 for root-update FLTrust on German (lower in 29 of 30 seeds) and 0.319 against 1.275 on Bail (26 of 30).

### *Which components produce the reduction*

Both methods use server data, so the empirical question is which of FU-Alignment's departures from FLTrust matter. A $2 \times 2$ factorial within the same protocol, using the cosine score in all four cells, varies the order of smoothing and rectification and the treatment of update magnitude (Table 5). Rectifying before smoothing raises adversarial weight by 0.035 on German and 0.024 on Bail, and rescaling updates to a common norm instead of clipping raises it by 0.077 and 0.027; all four paired confidence intervals exclude zero. Signed smoothing and clipping therefore both contribute. A paired run with the fairness term removed ($\alpha = 0$, all else equal) isolates the reference itself: at $\alpha = 0.1$ the fairness term lowers German adversarial weight by only 0.003 (Holm-adjusted $p = 0.011$) and changes no other endpoint detectably on either dataset. The reduction relative to root-update FLTrust is therefore attributable to how updates are processed and scored over time, not to the fairness direction at this weight.

### 4.4 RQ4: Adaptive-Access and Label-Protocol Boundaries

The adaptive-access study contains 320 runs over German and Bail, four methods, three attack schedules plus benign controls, and 10 paired seeds. Table 6 reports weight and influence over the rounds in which the adversaries poison; Section S3 of ESM 1 gives all-round means.

Reference evasion, the attack with the most knowledge of the server rule, is the most damaging one for FedAvg: its AUC falls to 0.590 on both datasets, against 0.661 and 0.749 in its benign runs. FU-Alignment gives the adversaries the lowest weight and influence of the four methods on both datasets (0.034 and 0.266 on German, 0.033 and 0.233 on Bail; root-update FLTrust 0.169 and 1.054, and 0.137 and 0.898) and keeps the highest attacked AUC (0.641 on German, 0.709 on Bail). On Bail, FU-Alignment also has the lowest processed influence under sustained and intermittent poisoning; under intermittent poisoning the GuardFed-style adaptation assigns slightly less weight (0.119 versus 0.128) but more influence (1.543 versus 0.660). On German, the GuardFed-style adaptation has the lowest weight and influence under both poisoning schedules.

The boundary is intermittent poisoning on German. There FU-Alignment gives the adversaries more weight and influence (0.210 and 1.237) than FedAvg (0.202 and 1.167) and root-update FLTrust (0.197 and 1.122), whereas under sustained poisoning its weight on the same dataset is 0.126. Each honest phase lets the adversaries rebuild a positive smoothed score that they then spend while poisoning; temporal memory, which removes a persistent opponent, can preserve the influence of one that alternates. The adaptive-access study therefore maps where the advantage holds and where it does not.

The known-label control tests benign utility under the corrected Pokec-z known-label protocol. FU-Alignment's paired AUC difference from FedAvg is $+0.002409$; the one-sided 95% bootstrap lower bound is $-0.002060$, above the prespecified $-0.01$ margin. FU-Alignment therefore preserves predictive utility on Pokec-z; its DPD and EOD differences from FedAvg are not significant (Table 7).

## 5 Discussion

***What report independence buys***

The audit separates two questions that are usually asked together. Whether a rule can be moved by what a client says about itself is a property of the aggregation interface, and the fixed-update intervention answers it without training noise. Whether a rule resists what a client sends is a property of the training dynamics, and it depends on the attack. FU-Alignment settles the first question by construction and, in the controlled study, also lowers the weight and processed influence of a fairness-poisoning client relative to root-update FLTrust. The ablation locates that advantage in how updates are processed: clipping before scoring and smoothing the signed score before rectification. Server anchoring alone, which FU-Alignment shares with FLTrust, does not produce it.

**Table 6 Adaptive-access boundary map.** $K = 10$ clients, two adversaries, 40 rounds, means over 10 paired seeds. $w_{\mathrm{adv}}$ and $I_{\mathrm{adv}}$ are averaged over the rounds in which the adversaries poison (from round 10 under sustained poisoning, alternate five-round blocks under intermittent poisoning, every round after the first under reference evasion); AUC, DPD and EOD are final endpoints. On Bail, FU-Alignment has the lowest processed influence under sustained and intermittent poisoning, and under reference evasion it gives the adversaries the lowest weight and influence of the four methods and keeps the highest AUC on both datasets. Intermittent poisoning on German is the boundary: FU-Alignment gives the adversaries more weight and influence than FedAvg and root-FLTrust. All-round means are in the Supplementary Information (ESM 1).

| Dataset | Attack | Rule | $w_{\mathrm{adv}}$ | $I_{\mathrm{adv}}$ | AUC | DPD | EOD |
|---|---|---|---|---|---|---|---|
| German | sustained | FedAvg | 0.202 | 1.163 | 0.646 | 0.146 | 0.148 |
| German | sustained | root-FLTrust | 0.196 | 1.122 | 0.652 | 0.155 | 0.106 |
| German | sustained | FU-Alignment | 0.126 | 0.749 | 0.641 | 0.080 | 0.088 |
| German | sustained | GuardFed-style | 0.009 | 0.054 | 0.661 | 0.097 | 0.080 |
| German | intermittent | FedAvg | 0.202 | 1.167 | 0.660 | 0.066 | 0.060 |
| German | intermittent | root-FLTrust | 0.197 | 1.122 | 0.664 | 0.046 | 0.063 |
| German | intermittent | FU-Alignment | 0.210 | 1.237 | 0.650 | 0.058 | 0.078 |
| German | intermittent | GuardFed-style | 0.022 | 0.146 | 0.653 | 0.066 | 0.062 |
| German | reference evasion | FedAvg | 0.202 | 3.272 | 0.590 | 0.219 | 0.232 |
| German | reference evasion | root-FLTrust | 0.169 | 1.054 | 0.624 | 0.167 | 0.187 |
| German | reference evasion | FU-Alignment | 0.034 | 0.266 | 0.641 | 0.060 | 0.077 |
| German | reference evasion | GuardFed-style | 0.049 | 0.656 | 0.629 | 0.073 | 0.064 |
| Bail | sustained | FedAvg | 0.191 | 1.549 | 0.738 | 0.023 | 0.026 |
| Bail | sustained | root-FLTrust | 0.173 | 1.114 | 0.718 | 0.016 | 0.034 |
| Bail | sustained | FU-Alignment | 0.093 | 0.491 | 0.716 | 0.008 | 0.028 |
| Bail | sustained | GuardFed-style | 0.129 | 1.708 | 0.714 | 0.010 | 0.031 |
| Bail | intermittent | FedAvg | 0.191 | 1.745 | 0.749 | 0.015 | 0.024 |
| Bail | intermittent | root-FLTrust | 0.172 | 1.098 | 0.722 | 0.014 | 0.026 |
| Bail | intermittent | FU-Alignment | 0.128 | 0.660 | 0.719 | 0.009 | 0.025 |
| Bail | intermittent | GuardFed-style | 0.119 | 1.543 | 0.724 | 0.012 | 0.028 |
| Bail | reference evasion | FedAvg | 0.191 | 60.738 | 0.590 | 0.064 | 0.071 |
| Bail | reference evasion | root-FLTrust | 0.137 | 0.898 | 0.642 | 0.045 | 0.061 |
| Bail | reference evasion | FU-Alignment | 0.033 | 0.233 | 0.709 | 0.015 | 0.036 |
| Bail | reference evasion | GuardFed-style | 0.048 | 2.762 | 0.665 | 0.016 | 0.031 |

**Table 7 Pokec-z known-label control supports AUC non-inferiority, not fairness superiority.** Mean ± sample SD over 10 paired seeds. FU-Alignment minus FedAvg: AUC +0.002409 (Wilcoxon $p = 0.375$), DPD +0.003431 ($p = 0.922$), and EOD +0.009312 ($p = 0.084$). The prespecified AUC non-inferiority margin is 0.01; the one-sided 95% bootstrap lower bound is −0.002060.

| Rule | AUC | DPD | EOD |
|---|---|---|---|
| FedAvg | 0.7421 ± 0.0079 | 0.0286 ± 0.0214 | 0.0245 ± 0.0203 |
| FU-Alignment | 0.7445 ± 0.0104 | 0.0321 ± 0.0257 | 0.0338 ± 0.0194 |

### *Deployment*

The rule needs a small labelled holdout with sensitive attributes at the server. In a cross-silo consortium of lenders or public agencies, such a set can be an audit sample curated by the consortium or held by a supervisor, and it can be kept in a trusted execution environment so that clients never see it. The server's extra work per round

is two gradient evaluations on the holdout and one inner product per client, and every round leaves a ledger entry that splits each client's score into utility and fairness parts. An auditor can therefore check after the fact why a client received its weight without access to any client's data. Two properties of the holdout shape what the reference can see. It is carved out before the clients are formed and keeps only the edges among its own nodes, so it is much sparser than the client graphs; on Bail most holdout nodes have no edge at all. The reference gradients are therefore driven mainly by node features, and the server never needs the clients' graph structure. The holdout is also drawn from the same validation distribution as the evaluation split, the premise that FLTrust makes for its root dataset.

***Limitations***

Report independence closes one input channel; it does not provide general Byzantine robustness. An adversary whose update aligns with the server reference keeps influence, and an adversary that alternates honest and poisoning phases can exploit the memory of the smoothed score, as intermittent poisoning on German shows. The rule assumes that the holdout is clean, confidential and representative of the federation; a holdout drawn from a shifted population, server compromise and Sybil participation are not tested here. Anchoring to a 200-node holdout has a benign cost on Bail, where FU-Alignment's clean AUC is below FedAvg's, a cost shared by root-update FLTrust. The fairness term of the reference has no detectable effect at the weight used here, so the study does not show that the fairness direction itself improves allocation or fairness; whether a larger weight trades utility for fairness is untested. The privacy analysis covers only the released report channel under a Gaussian approximation and gives no guarantee for model updates. Finally, the three graphs have binary sensitive attributes; intersectional groups, other graph domains and larger federations remain to be studied.

# 6 Conclusion

Aggregation rules that weight clients by what they report about themselves can be steered through those reports. A fixed-update audit makes this measurable: with updates and server state held fixed, every report-dependent rule tested shifts weight towards a client that falsifies its report, while rules that read no report produce identical aggregates.

FU-Alignment removes this channel by scoring clipped updates against a reference gradient that the server computes on its own holdout. Under controlled fairness poisoning it gives the adversary less weight and less processed influence than root-update FLTrust on both datasets, with higher AUC, and a factorial ablation traces the advantage to clipping before scoring and to smoothing the signed score before rectification. Under reference evasion it gives the adversaries the lowest weight and influence of the four rules compared and keeps the highest AUC, and on Pokec-z it preserves the predictive utility of FedAvg. Every round leaves a ledger that splits each client's score into utility and fairness parts, so allocation decisions can be audited after the fact.

The adaptive-access study and Section 5 delimit where these results hold. Natural next steps are holdouts drawn from shifted populations, attackers that alternate phases against the score memory, and federations with more clients and intersectional sensitive groups.

**Supplementary information.** The online version contains Supplementary Information (ESM 1): the report-channel privacy analysis with proofs, the complete adaptive-access grid averaged over all rounds, the GuardFed-style tuning protocol, and a supporting stealth-poisoning sweep.

## Declarations

**Funding.** No external funding was received for conducting this study.

**Competing interests.** The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.

**Ethics approval.** Not applicable. This study does not involve human participants or live vertebrate animals. All datasets analyzed are publicly available standard machine learning benchmarks.

**Consent to participate.** Not applicable.

**Consent for publication.** Not applicable.

**Data availability.** All datasets analysed in this study are public. German Credit and Bail Recidivism were obtained from the NIFTY benchmark release [17] (https://github.com/chirag126/nifty); Pokec-z was obtained from the FairGNN release [12] (https://github.com/EnyanDai/FairGNN), derived from the Pokec social network [43]. Preprocessing, the server-holdout split, and the client partitions are generated by scripts in the code repository from fixed seeds.

**Code availability.** The TrustFedGNN source code, experiment runners, per-seed result artifacts, and the verifier that checks them are available at https://github.com/ngngsonan/TrustFedGNN. Each result in this article is identified by its artifact ID (Table 2).

**Author contributions.** N.-S.-A. Nguyen: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft. Q.-V. Dang: Supervision, Validation, Writing – review & editing. T.-N. Huynh: Supervision, Validation, Writing – review & editing. All authors read and approved the final manuscript.

## References

[1] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations (ICLR), pp. 1–14 (2017). https://openreview.net/forum?id=SJU4ayYgl

[2] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: International Conference on Learning Representations (ICLR), pp. 1–12 (2018). https://openreview.net/forum?id=rJXMpikCZ

[3] McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: Artificial Intelligence and Statistics (AISTATS), vol. 54, pp. 1273–1282 (2017). https://proceedings.mlr.press/v54/mcmahan17a.html

[4] Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., *et al.*: Advances and open problems in federated learning. Foundations and Trends® in Machine Learning **14**(1–2), 1–210 (2021) https://doi.org/10.1561/2200000083

[5] Ezzeldin, Y.H., Yan, S., He, C., Ferrara, E., Avestimehr, A.S.: FairFed: Enabling group fairness in federated learning. In: AAAI Conference on Artificial Intelligence, vol. 37, pp. 7494–7502 (2023). https://doi.org/10.1609/aaai.v37i6.25911

[6] Li, T., Sanjabi, M., Beirami, A., Smith, V.: Fair resource allocation in federated learning. In: International Conference on Learning Representations (ICLR), pp. 1–17 (2020). https://openreview.net/forum?id=ByexElSYDr

[7] Cui, N., Wang, X., Wang, W.H., Chen, V., Ning, Y.: Equipping federated graph neural networks with structure-aware group fairness. In: IEEE International Conference on Data Mining (ICDM), pp. 980–985 (2023). https://doi.org/10.1109/ICDM58522.2023.00111

[8] Khan, K.: FedGraph-Fair: Federated learning with personalization and fairness via dynamic graphs and DRO. Information Sciences **728**, 122710 (2026) https://doi.org/10.1016/j.ins.2025.122710

[9] Bendoukha, A.-A., Demirag, D., Kaaniche, N., Boudguiga, A., Sirdey, R., Gambs, S.: Towards privacy-preserving and fairness-aware federated learning framework. Proceedings on Privacy Enhancing Technologies (PoPETs) **2025**(1), 845–865 (2025) https://doi.org/10.56553/popets-2025-0044

[10] Dang, Q.-V., Nguyen, N.-S.-A.: FedFairGNN: A privacy-preserving and fairness-aware federated graph neural network for fraud detection. In: Proceedings of IndabaX Nigeria 2026: Building Scalable AI That Works: From Research to Deployment in Resource-Constrained Environments. Proceedings of Machine Learning Research, vol. 319, pp. 74–86 (2026). https://proceedings.mlr.press/v319/dang26a.html

[11] Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: Network and Distributed System Security Symposium (NDSS), pp. 1–18 (2021). https://doi.org/10.14722/ndss.2021.24434

[12] Dai, E., Wang, S.: Say no to the discrimination: Learning fair graph neural networks with limited sensitive attribute information. In: ACM International Conference on Web Search and Data Mining (WSDM), pp. 680–688 (2021). https://doi.org/10.1145/3437963.3441752

[13] Yang, C., Liu, J., Yan, Y., Shi, C.: FairSIN: Achieving fairness in graph neural networks through sensitive information neutralization. In: AAAI Conference on Artificial Intelligence, vol. 38, pp. 9241–9249 (2024). https://doi.org/10.1609/aaai.v38i8.28776

[14] Li, Z., Dong, Y., Liu, Q., Yu, J.X.: Rethinking fair graph neural networks from re-balancing. In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 1736–1745 (2024). https://doi.org/10.1145/3637528.3671826

[15] Zhu, Y., Li, J., Bian, Y., Zheng, Z., Chen, L.: One fits all: Learning fair graph neural networks for various sensitive attributes. In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 4689–4699 (2024). https://doi.org/10.1145/3637528.3672029

[16] Dong, Y., Liu, N., Jalaian, B., Li, J.: EDITS: Modeling and mitigating data bias for graph neural networks. In: The Web Conference (WWW), pp. 1259–1269 (2022). https://doi.org/10.1145/3485447.3512173

[17] Agarwal, C., Lakkaraju, H., Zitnik, M.: Towards a unified evaluation of fair graph neural networks. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 17865–17877 (2021)

[18] Lee, Y.-C., Shin, H., Kim, S.-W.: Disentangling, amplifying, and debiasing: Learning disentangled representations for fair graph neural networks. In: AAAI Conference on Artificial Intelligence, vol. 39, pp. 12013–12021 (2025). https://doi.org/10.1609/aaai.v39i11.33308

[19] Park, S., Lim, S.: FnRGNN: Distribution-aware fairness in graph neural network. In: ACM International Conference on Information and Knowledge Management (CIKM), pp. 1–10 (2025). https://doi.org/10.1145/3746252.3760796

[20] Wang, Z., Yan, X., Jin, Y.: Fairness-oriented vertical federated GNNs with incomplete sensitive attributes. Information Fusion **127**, 103871 (2026) https://doi.org/10.1016/j.inffus.2025.103871

[21] Zhou, Z., Yang, S., Zhao, F., Ren, X.: FairGFL: Privacy-preserving fairness-aware federated learning with overlapping subgraphs. IEEE Transactions on Parallel and Distributed Systems **37**, 710–725 (2026) https://doi.org/10.1109/TPDS.2025.3649863

[22] Corbucci, L., Heikkilä, M.A., Solans Noguero, D., Monreale, A., Kourtellis, N.: PUFFLE: Balancing privacy, utility, and fairness in federated learning. In:

European Conference on Artificial Intelligence (ECAI). Frontiers in Artificial Intelligence and Applications, vol. 392, pp. 1639–1647 (2024). https://doi.org/10.3233/FAIA240671

[23] Zhang, L., Han, Z., Feng, X., Zhang, J., Li, Y., Chen, C.: FedFACT: A provable framework for controllable group-fairness calibration in federated learning. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 1–19 (2025). https://arxiv.org/abs/2506.03777

[24] Dwork, C., Roth, A.: The algorithmic foundations of differential privacy. Foundations and Trends in Theoretical Computer Science **9**(3–4), 211–487 (2014) https://doi.org/10.1561/0400000042

[25] Mironov, I.: Rényi differential privacy. In: IEEE Computer Security Foundations Symposium (CSF), pp. 263–275 (2017). https://doi.org/10.1109/CSF.2017.11

[26] Ling, X., Fu, J., Wang, K., Li, H., Cheng, T., Chen, Z.: FedFDP: Fairness-Aware Federated Learning with Differential Privacy. arXiv preprint arXiv:2402.16028 (2024). https://arxiv.org/abs/2402.16028

[27] Bagdasaryan, E., Poursaeed, O., Shmatikov, V.: Differential privacy has disparate impact on model accuracy. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32, pp. 15453–15462 (2019). https://arxiv.org/abs/1905.12101

[28] Cummings, R., Gupta, V., Kimpara, D., Morgenstern, J.: On the compatibility of privacy and fairness. In: Adjunct Publication of the 27th Conference on User Modeling, Adaptation and Personalization (UMAP), pp. 309–315 (2019). https://doi.org/10.1145/3314183.3323847

[29] Tran, C., Dinh, M.H., Fioretto, F.: Differentially private empirical risk minimization under the fairness lens. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 27555–27565 (2021). https://arxiv.org/abs/2106.02674

[30] Fioretto, F., Tran, C., Van Hentenryck, P., Zhu, K.: Differential privacy and fairness in decisions and learning tasks: A survey. In: International Joint Conference on Artificial Intelligence (IJCAI-ECAI), Survey Track, pp. 5473–5481 (2022). https://arxiv.org/abs/2202.08187

[31] Yin, D., Chen, Y., Ramchandran, K., Bartlett, P.L.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: International Conference on Machine Learning (ICML), vol. 80, pp. 5626–5635 (2018). https://proceedings.mlr.press/v80/yin18a.html

[32] Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural

Information Processing Systems (NeurIPS), pp. 119–129 (2017)

[33] El Mhamdi, E.M., Guerraoui, R., Rouault, S.: The hidden vulnerability of distributed learning in byzantium. In: International Conference on Machine Learning (ICML), vol. 80, pp. 3521–3530 (2018). https://proceedings.mlr.press/v80/mhamdi18a.html

[34] Fung, C., Yoon, C.J., Beschastnikh, I.: Mitigating Sybils in Federated Learning Poisoning. arXiv preprint arXiv:1808.04866 (2018)

[35] Nguyen, T.D., Rieger, P., Chen, H., Yalame, H., M"ollering, H., Fereidooni, H., Marchal, S., Miettinen, M., Mirhoseini, A., Sadeghi, A.-R., Schneider, T.: FLAME: Taming backdoors in federated learning. In: 31st USENIX Security Symposium (USENIX Security 22), pp. 1415–1432 (2022)

[36] Meerza, S.I.A., Liu, J.: EAB-FL: Exacerbating algorithmic bias through model poisoning attacks in federated learning. In: International Joint Conference on Artificial Intelligence (IJCAI), pp. 458–466 (2024). https://doi.org/10.24963/ijcai.2024/51

[37] Kasyap, H., Fang, M., Liu, Z., Maple, C., Tripathy, S.: Fairness-Constrained Optimization Attack in Federated Learning. arXiv preprint arXiv:2510.12143 (2025). https://arxiv.org/abs/2510.12143

[38] Xu, X., Lyu, L., Ma, X., Miao, C., Foo, C.S., Low, B.K.H.: Gradient driven rewards to guarantee fairness in collaborative machine learning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 16104–16117 (2021)

[39] Li, Y., Zhou, Y., Guo, Z., Yang, N., Zhang, Y., Chen, H., Yuan, D., Ding, W., Pedrycz, W.: GuardFed: A trustworthy federated learning framework against dual-facet attacks. arXiv preprint arXiv:2511.09294 (2025)

[40] Shapley, L.S.: A value for $n$-person games. In: Contributions to the Theory of Games (AM-28), Volume II, pp. 307–318. Princeton University Press, Princeton, NJ (1953). https://doi.org/10.1515/9781400881970-018

[41] Liu, Z., Chen, Y., Yu, H., Liu, Y., Cui, L.: GTG-Shapley: Efficient and accurate participant contribution evaluation in federated learning. ACM Transactions on Intelligent Systems and Technology (TIST) **13**(4), 1–21 (2022)

[42] Tastan, N., Fares, S., Aremu, T., Horvath, S., Nandakumar, K.: Redefining contributions: Shapley-driven federated learning. In: International Joint Conference on Artificial Intelligence (IJCAI), pp. 8200–8208 (2024). https://arxiv.org/abs/2406.00569

[43] Takac, L., Zabovsky, M.: Data analysis in public social networks. In: International Scientific Conference and International Workshop Present Day Trends of Innovations, pp. 1–6 (2012)